Assignment 11: Parallel Processing and Pipeline Hazards

Auqib Hamid Lone
Department of Computer Science & Engineering, GCET Ganderbal

September 5, 2026

This assignment follows the Week 11 treatment of parallel organization and instruction pipelining in Hamacher, Ch. 5 and Ch. 10, and Stallings, §16.4 and §20.1 [1, 2].
Answer every question. Show all working for Numerical and Design questions.

11.1 Conceptual

Q1. Classify each organization as SISD, SIMD, MISD, or MIMD. For each answer, justify the classification by naming the instruction streams and data streams rather than counting hardware units alone.

(a) One scalar core executes one instruction sequence to sum one scalar array.

(b) One vector instruction adds corresponding elements of two eight-element vectors.

(c) Four cores execute independent instruction sequences on four independent matrix tiles.

(d) Several independent filters inspect the same sensor stream; state the strict stream-based classification and the practical implementation described in the lecture.

Q2. A five-stage pipeline has one shared memory port. In one cycle, an instruction in MEM requests a data access while the next instruction in IF requests an instruction fetch. In two separate cases, a dependent ADD needs a preceding ALU result and a dependent ADD needs a preceding load result. Classify each situation as structural, data/RAW, or control, and state the cheapest correct response described by the lecture. Explain why forwarding does not remove the load-use delay.

11.2 Numerical

Q3. Five independent instructions execute in a balanced five-stage pipeline with one cycle per stage and no hazards.

(a) Draw the complete timing table for cycles 1 through 9 using the stages IF, ID, EX, MEM, and WB.

(b) Compute the sequential time, ideal pipelined time, and finite- stream speedup.

(c) State the large-stream ideal speedup limit and distinguish it from the latency of one instruction.

Q4. Consider the following three-instruction stream in the same five-stage model:

`LW R2,4(R3); ADD R5,R2,R6; SUB R7,R5,R8`.

Assume load data is available after MEM, the dependent ALU operation needs it at the start of EX, and ALU results can be forwarded. Draw the timing table for cycles 1 through 8, mark every stall or bubble, and explain which value is forwarded on which dependence.

11.3 Design

Q5. A workload has branch frequency $f_{br} = 0.25$, misprediction rate $p_{miss} = 0.04$, and a flush penalty $P_{flush} = 3$ cycles. Compute the approximate branch contribution ΔCPI. Then recompute it when a deeper pipeline raises the flush penalty to 5 cycles. State the throughput trade-off that the two numbers reveal.

Q6. Use the running stream

`LW R1,0(R2)`; `ADD R3,R1,R4`; `SUB R5,R3,R6`; `BEQ R5,R0,L`; `OR R7,R8,R9`.

For each adjacent relationship, identify the dependency or hazard, state the stage/resource/value that creates the problem, and choose a resolution from stalling, forwarding, scheduling, prediction, or flushing. Identify which relationship requires one load-use bubble, which can use ALU forwarding, and which may invalidate younger instructions after a branch. Conclude with one trade-off among throughput, latency, area, power, complexity, or recovery cost.

References

[1] V. Carl Hamacher, Zvonko G. Vranesic, Safwat G. Zaky, and Naraig Manjikian. *Computer Organization and Embedded Systems*. McGraw-Hill, 6th edition, 2012. ISBN 978-0-07-338065-0. Bib key retained as Hamacher2002_computer_organization for citation-key stability across existing lectures; the file indexed under master_supporting_docs/CS401/supporting_books/Hamacher2002/ and page-cited by Week 8 is this 6th edition, not the 5th ed. (2002) the key name suggests – see that folder's index.md Edition notice, 2026-08-11.

[2] William Stallings. *Computer Organization and Architecture: Designing for Performance*. Pearson, 10th edition, 2015.